

Architecture logicielle & systèmes distribués

[Accueil](#)[Introduction](#)[Jour 1](#)[Jour 2](#)[Jour 3](#)[Jour 4](#)[Jour 5](#)[Jour 6](#)[TP1](#)[TP2](#)[TP3](#)[Exercices](#)[Corrigés](#)[Glossaire](#)[Live Coding](#)[Dossier Architecture](#)[FAQ](#)[Support HTML modulaire](#)[Master 2](#)[6 jours](#)[3 TP progressifs](#)[Java / Spring Boot](#)

Corrigés et éléments d'évaluation

Objectifs pédagogiques

Fournir des pistes de corrigé et aider à l'évaluation des étudiants.

Temps estimé

≈ 6h incluant démos et quiz

Prérequis

Exercices complétés

Table des matières

- **Éléments de corrigé**
- Barème

Éléments de corrigé

Ces corrigés donnent une direction attendue. En architecture, plusieurs réponses sont acceptables si elles sont justifiées avec rigueur.

▼ Exercice : pourquoi garder un monolithe ?

Parce qu'une équipe réduite, un domaine encore mouvant et une faible maturité d'exploitation rendent souvent le monolithe modulaire plus rentable. Le vrai critère est le coût global d'évolution, pas la mode.

▼ Exercice : forte charge sur le catalogue

Extraire ou isoler le catalogue si nécessaire, mettre en place cache, scaling horizontal ciblé et éventuelle réplication lecture, tout en gardant la gestion des prêts séparée.

▼ Exercice : risques d'un appel REST critique

Latence, timeouts, erreurs transitoires, propagation de panne, surcharge en cas de retries, rupture de contrat.

▼ Exercice : quand préférer un événement

Quand le consommateur n'a pas besoin de répondre immédiatement et qu'on veut un découplage temporel, par exemple notifications, audit, analytics.

Conseils de barème enseignant

Axe	Barème conseillé
Compréhension des concepts	30%
Justification des choix	25%
Qualité du code / structure	20%
Gestion des risques et limites	15%
Qualité du dossier d'architecture	10%

[← Exercices](#)[Glossaire →](#)